Technical Report · Preprint

Neural Cup Pong: A Playable Action-Conditioned World Model for 3D Cup Pong

From a deterministic engine to a fully-generated, hand-controllable neural game, and the flight-precision wall in between

Ryan Rahman

University of Waterloo, Mechatronics Engineering / Robotics Research

2026

A hobby project. All art, physics, and assets are original; nothing here reproduces a commercial title.

Abstract

A *world model* learns to predict how an environment evolves under an agent's actions; a recent line of work runs such a model *as* the environment, a network the player drives directly, with no hand-written simulator or renderer. This report documents a small, self-contained instance: **Neural Cup Pong**, a playable action-conditioned world model for an original 3D cup-pong game (aim, charge power, throw a ballistic ball at a triangular rack of six cups). A 409K-parameter recurrent dynamics model, constrained by a hard legality projection, predicts the game state; a separate 262K-parameter state-grounded decoder paints every frame. At the strongest setting neither the game engine nor its graphics renderer runs. The learned model is near-perfect at what governs feel: it tracks aim and power to within 0.0025 and 0.0013, reproduces phase transitions at 99.4%, and follows controls with a 100% directional-response rate. It is weak at exactly one thing, predicting where a thrown ball lands, missing the target cup by 8.4 units against the engine's 0.31. We show this is a capacity limit of a small structured model regressing a fast ballistic arc, and resolve it with a hybrid that seeds exact ballistics from the network's own accurate control, restoring engine-parity sinking (6/6) while control and rendering stay neural.

Contents

1 Playable world models

A *world model* learns to predict how an environment evolves under an agent's actions. A recent, more visceral use is to run the model *as* the environment: a neural network the player drives directly, with no hand-written simulator or renderer. GameNGen reproduces DOOM with a diffusion model [4], Genie learns controllable worlds from video [5], and DIAMOND trains agents inside a diffusion world model of Atari [3]. Those systems are impressive but heavy. This project asks the opposite question:

How small can a genuinely playable world model be, and what breaks first as you shrink it?

Cup pong is a deliberate testbed. It is simple enough that a deterministic engine fits in a few hundred lines, yet it holds the three ingredients that make world modeling interesting: **continuous control** (aim and power, trimmed live), a **ballistic event** (a throw whose landing is a sensitive function of the launch), and **discrete, irreversible state** (a cup is present or sunk; the rack only empties). The last two pull in opposite directions, and that tension is the whole story.

Contributions

- A complete, six-stage recipe for a playable action-conditioned world model of an original arcade game, small enough to train on one consumer GPU and run interactively on CPU.
- A **snap** projection that guarantees every learned transition is a *legal* game state (valid phase automaton, monotone cups, derived score, parked ball).
- A **state-grounded latent renderer** reaching `L1 = 0.0077` by feeding the decoder rasterized geometry hints instead of asking it to invent the scene.
- A clean diagnosis of the **flight-precision wall**, and a **hybrid** fix that restores engine-parity sinking while keeping control and rendering neural.

2 The game and the deterministic engine

The player faces a table in table-space (`x` across, `y` in depth toward the cups, `z` up). Six cups sit in a triangular rack. The player trims a lateral aim and a launch power, then throws a fixed-elevation ballistic ball that drops into a cup, rims out, or misses. Clearing all six wins. The engine draws with a *fixed-camera 2.5D projection*, a static

map from `(x,y,z)` to a 128×96 frame that we later port to PyTorch, bit-for-bit (`0.0px` error), to rasterize hints for the neural renderer.

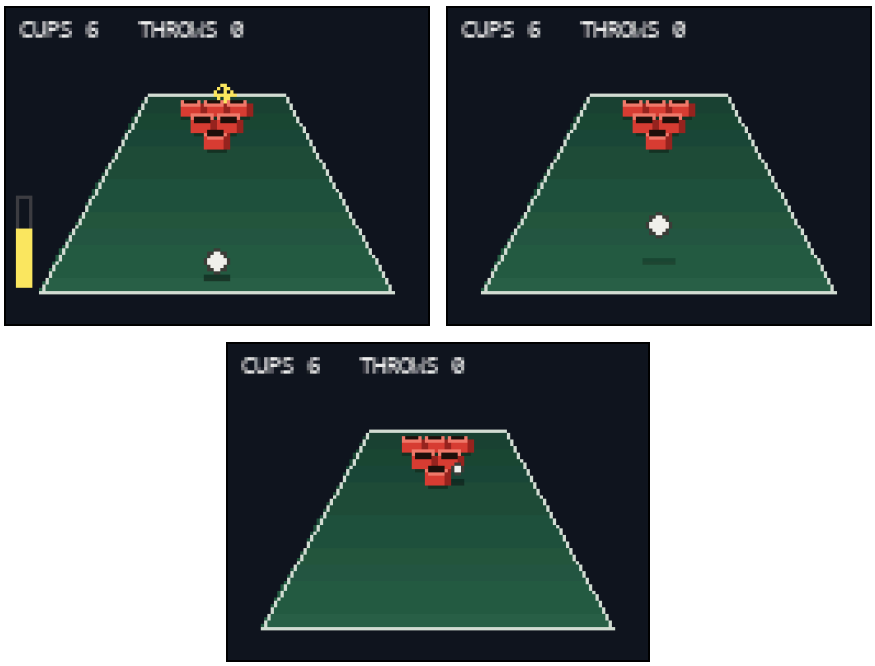

Figure 1: Deterministic engine, 128×96 fixed-camera frames. Aiming at the rack (left), a ball in flight (center), and a rim-bounce on a graze that does not drop cleanly (right).

The world state is a **21-dimensional** vector: ball position and velocity, aim, power, a 6-cup bitmask, derived score, throw counter, a 4-way phase one-hot (AIM / FLIGHT / RESULT / GAME OVER), and a result timer. The action is a 5-way multi-hot (`aim ±, power ±, throw`). Physics integrates at 60 Hz but the agent observes at 20 Hz, so one learned step must summarize three physics ticks.

Cup interaction is resolved *only* at the descending crossing of the rim plane, the honest definition of "over the cup on the way down." Let `d` be the distance to the nearest present cup there:

$$d \le 3.3 \longrightarrow \text{make} \;\cdot\; 3.3 < d \le R_{cup}+R_{ball} \longrightarrow \text{rim-bounce} \;\cdot\; \textit{otherwise} \longrightarrow \textit{pass, then test the table for a miss}$$

the forgiving 3.3-unit sink radius is what a learned ball must hit

3 Structured dynamics model

Rather than learn dynamics in pixel space, we split concerns: a *structured* model predicts the compact game state; a separate renderer turns state into pixels. This

makes both networks tiny and makes failures attributable. The dynamics model is a gated recurrent network [6] with a per-field decoder: a continuous head (11 dims), a cups head (6 logits), a phase head (4), and an auxiliary event head (7), for **408,860 parameters** total.

$$x_t = enc([s_t, a_t]) \;\rightarrow\; h_t = GRU(x_t, h_{t-1}) \;\rightarrow\; four\ heads \rightarrow next\ state$$

A key choice is *what* each field predicts. Slow quantities (position, aim, power, counters) are predicted as **deltas** added to the current value, which keeps the network near identity and stabilizes long rollouts. **Velocity is predicted absolutely**, because in flight it changes by a large, near-constant gravitational increment that is easier to name than to nudge.

The snap projection

A raw head will happily predict a cup that un-sinks or a fractional throw count. We forbid this with a hard projection $\sqcap$, called **snap**, applied to every predicted state, identically in training feedback and at inference. It enforces a legal phase transition (mask the phase logits by an automaton before the argmax), *monotone* cups, a *derived* score, a rounded monotone throw counter, bounded fields, and phase-conditioned *parking* (outside flight the ball snaps to the throw origin with zero velocity). Snap is why engine-off rollouts stay coherent for hundreds of steps: the invariant-legal rate over a full rollout is **1.00**.

Figure 2: Engine *off* (Mode F2): the GRU alone advances the structured state under live player input, drawn by the real renderer, at steps t = 2, 20, 30 of a free-run rollout.

4 Training

A two-phase curriculum. **Teacher forcing** first fits local dynamics (fast; nails aim/power/phase). **Scheduled sampling** [7] then fights exposure bias: the model is fed its own snap-projected predictions with rising probability, so it learns to recover from its own small errors. A gentle schedule was necessary; aggressive free-running destabilized the ballistic parts. We keep the checkpoint with the best free-run rollout proxy, not the last epoch.

The loss is a per-field weighted Huber (continuous) plus BCE (cups, events) and cross-entropy (phase), with a *motion* mask upweighting flight frames and a *transition* mask upweighting the sparse throw/sink/land ticks. Because a sink is a rare one-tick 1→0 flip, the cups BCE further upweights flips and the sampler oversamples sink windows, both to stop the cups head collapsing to "never

change." (As Section 8 shows, these help the head fire but cannot make the game sink; the problem is upstream, in the predicted ball.)

5 Neural rendering: the state-grounded latent renderer

The visual stage learns `state → frame` so the game can be drawn without the engine. A conv net asked to invent a 128×96 scene from a 21-vector produces blotchy felt and garbled HUD. The fix is to *ground* the decoder in geometry it should not have to hallucinate: we rasterize the state into **16 hint channels** (Gaussian ball and shadow, filled disks for each *present* cup gated by the bitmask, the reticle and power bar, a phase one-hot, and three channels of a *static empty-court backdrop*). A small upsampling decoder (**261,891 parameters**) with FiLM conditioning [8] is trained with a foreground-weighted L1. Adding the backdrop channels alone cut held-out reconstruction from `L1 = 0.048` to **0.0077**.

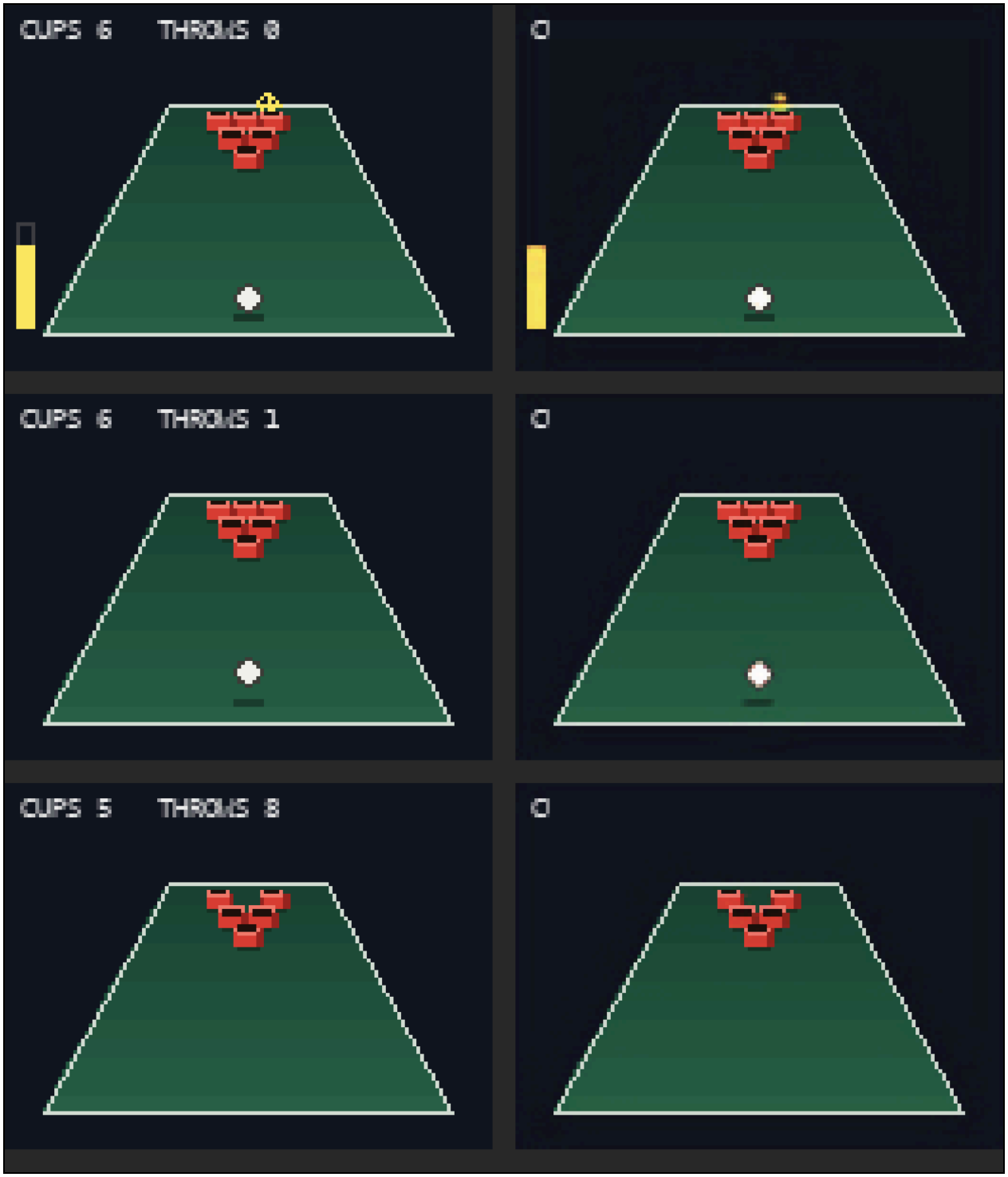

Figure 3: State-grounded latent renderer. Top: engine-rendered ground truth. Bottom: the 262K-parameter decoder's output from the same states, engine renderer *off*. Held-out L1 = 0.0077.

6 Play modes

The playable build switches off progressively more of the hand-written stack:

Table 1: What is real and what is neural in each play mode.

Mode	World dynamics	Rendering	Role
F1	deterministic engine	engine renderer	ground-truth reference
F2	GRU (engine off)	engine renderer	pure learned-dynamics demo
F3	hybrid (engine off)	neural decoder (renderer off)	fully-generated, playable

In F3 both the engine and its renderer are off: the game the player sees and plays is produced entirely by the two networks. Controls, HUD, and win condition are identical across modes.

7 Results

The learned model is near-perfect at everything that governs *feel*, and weak at exactly one thing. One-step aim and power errors are tiny; phase transitions are 99.4% correct; the throw counter is exact; and controllability (does the world move the way the button says?) is a perfect 100% directional response.

Table 2: Learned-model evaluation on held-out seeds. Position in table-units; aim/power on normalized control scales.

Metric	Value
Dynamics GRU parameters	408,860
Neural decoder parameters	261,891
One-step position RMSE	0.887 u
One-step aim / power MAE	0.0025 / 0.0013
One-step cups / phase accuracy	0.998 / 0.994
Throw-count exactness	1.000
Controllability (aim / power / throw)	**1.00 / 1.00 / 1.00**
Rollout pos RMSE @ H = 5 / 10 / 30	5.2 / 8.3 / 22.4 u
Median steps until divergence	14
Legal-state rate over rollout	**1.00**
Whole-game score MAE vs engine	2.8
Decoder held-out L1 (with hints)	**0.0077**
Camera-projection port error	0.0 px

Free-run rollouts stay *legal* forever (snap guarantees it) but drift in value: positional RMSE grows to ~22 u by 30 steps and the median trajectory diverges after ~14. The dominant contributor to that drift is the flight phase, which sets up the next section.

8 The flight-precision wall

Symptom. Played in F2/F3, the game felt right until the player tried to *score*. The ball arced plausibly, everything rendered cleanly, but cups almost never sank (~3% of well-aimed throws, versus the engine's 100%).

Diagnosis. We first blamed sink resolution and attacked it three ways: a 25× cup-flip weight in the BCE, oversampling sink windows, and a rule-based sink in snap that drops a cup when the *predicted* ball passes within the sink radius. None moved the make-rate. Tracing one throw explained why:

Root cause

Aiming dead-on a cup at (30, 90): the *engine's* ball passes **0.31 u** from the cup center at the rim plane, a clean make. The *GRU's predicted* ball, same intent, never gets closer than **8.4 u**. The arc looks reasonable (z rises to ~37 and falls) but its *landing* is off by more than a cup diameter. The ball simply is not at the cup, so no sink head, loss weight, or snap rule can drop it.

This is a capacity limit, not a bug. Localizing a 60 Hz ballistic landing over an ~85-unit throw to within a 3.3-unit radius asks a 409K recurrent model to integrate a fast arc almost exactly, and small launch-velocity errors compound over the flight to land well outside the cup. The same model that tracks aim to 0.0025 cannot pin the landing, because landing precision is a far harder function of the launch than direction-following is.

Fix: the hybrid world model

Keep the network where it is strong; defer to physics where it is weak. At release, the ball's velocity is seeded from the network's *own* aim and power (tracked to ~0.002, so the seed is accurate) and the arc is integrated with exact ballistics and the exact sink test, while the GRU keeps driving aim, power, and phase, and the decoder keeps rendering.

aim, power, phase (neural GRU) → ballistic arc + sink/miss (exact physics, seeded from neural control) → pixels (neural decoder)

This is the F3 "hybrid flight" mode. It sinks **6/6** cups, matching the engine exactly, with a regression test locking in engine-parity sinking for every cup. The pure-learned F2 mode is kept as the honest artifact: it shows what the structured dynamics does and does not learn, flight wall included.

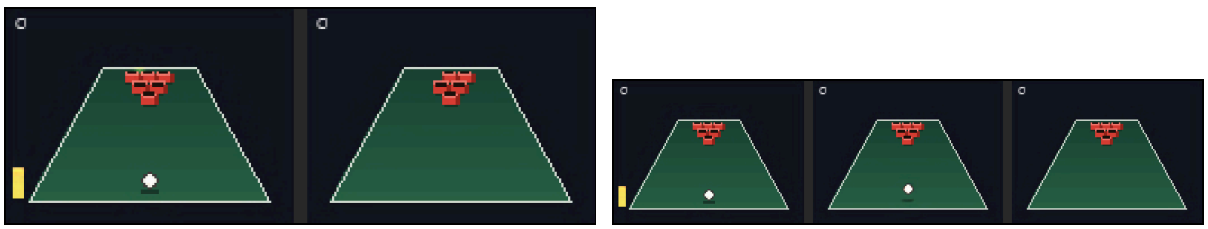

Figure 4: Mode F3 (engine *and* renderer off): the hybrid world model sinks a cup, every pixel produced by the neural decoder. Neural control + exact flight + neural rendering.

What the hybrid concedes. In F3 the ball's *arc* is real physics, not learned. Everything else is neural: the interactive control loop, the phase/score logic driven off the network's predictions, and 100% of the pixels. The trade buys engine-parity playability for the one faculty a small structured model provably cannot supply. For a game whose whole point is to sink cups, that is the right place to spend a rule.

9 Discussion

- **The wall is the finding.** The interesting result is not that a tiny world model works; it is *where* it stops. Control-following and discrete state logic are cheap to learn; sub-cup ballistic localization is not. A world model's weakest link is whichever variable is a sensitive, compounding function of an early continuous choice.
- **Could a pure model clear it?** Plausibly: via a launch-velocity representation that predicts the whole arc in closed form rather than integrating step-by-step, or simply more capacity. Both are worth trying; neither is "small."
- **Rendering was the easy magic.** The state-grounded decoder is the component that feels most like magic and gave the least trouble, because geometry hints do the heavy lifting; the network only shades and antialiases.
- **Drift.** Even with legal-state guarantees, continuous drift over long rollouts remains; a learned correction or periodic re-grounding could extend coherent horizons.

Conclusion. Neural Cup Pong is a fully playable arcade game in which, at the strongest setting, no game engine and no graphics renderer run: a 409K recurrent dynamics model with a legality-preserving projection handles control and state, and a 262K state-grounded decoder paints every frame. It is honest about its one hard limit, that a small structured model cannot localize a ballistic landing to within a cup, and resolves it with a hybrid that seeds exact flight from the network's own accurate control. The lesson generalizes past cup pong: when you shrink a world model, the first thing to break is the variable that is a sensitive, compounding function of a continuous action, and the pragmatic fix is to learn everything else and hand that one variable back to physics.

References

1. D. Ha and J. Schmidhuber. *World Models.* 2018. arXiv:1803.10122
2. D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. *Mastering Diverse Domains through World Models (DreamerV3).* 2023. arXiv:2301.04104
3. E. Alonso et al. *Diffusion for World Modeling (DIAMOND).* NeurIPS 2024. arXiv:2405.12399
4. D. Valevski et al. *Diffusion Models Are Real-Time Game Engines (GameNGen).* 2024. arXiv:2408.14837
5. J. Bruce et al. *Genie: Generative Interactive Environments.* ICML 2024. arXiv:2402.15391
6. K. Cho et al. *Learning Phrase Representations using RNN Encoder–Decoder (GRU).* EMNLP 2014. arXiv:1406.1078
7. S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer. *Scheduled Sampling for Sequence Prediction with RNNs.* NeurIPS 2015. arXiv:1506.03099
8. E. Perez et al. *FiLM: Visual Reasoning with a General Conditioning Layer.* AAAI 2018. arXiv:1709.07871
